Ocelot API Gateway Configuration – Complete Explanation

What is Ocelot API Gateway?

Introduction

Ocelot is an **open-source API Gateway framework built for ASP.NET Core**. It acts as a **single entry point for client applications** and sits between clients and microservices. **Instead of clients communicating directly with individual services, they send requests to Ocelot**, and Ocelot forwards those requests to the appropriate downstream services.

Ocelot simplifies communication in a microservices architecture by providing features such as **routing, authentication, authorization, rate limiting, caching, service discovery, load balancing, request aggregation, and logging**.

Why Do We Need an API Gateway?

In a microservices architecture, an application may consist of multiple independent services such as User Service, Product Service, Order Service, and Payment Service. Without an API Gateway, clients must communicate with each service directly.

Ocelot NuGet Packages

Core Ocelot Package

The primary package required to use Ocelot API Gateway is:

Install-Package Ocelot

or

dotnet add package Ocelot

This package provides the core API Gateway functionality

Configuration Package

To load configuration from JSON files, use the built-in ASP.NET Core configuration providers.

Typically, no additional package is required because ASP.NET Core includes these packages by default.

Example:

```
builder.Configuration
       .AddJsonFile("ocelot.json", optional: false, reloadOnChange: true);
```

Configuration Introduction

This Ocelot configuration acts as an API Gateway for multiple microservices. It provides a single entry point for clients and routes requests to the appropriate downstream services.

In this configuration, there are three routes and one aggregate route. The routes are configured for retrieving all users, retrieving a specific user by ID, and retrieving weather information.

An aggregate route combines responses from multiple services and returns them as a single response to the client.

```
{
 "Routes": [
  {
   "Key": "userService",
   "UpstreamPathTemplate": "/api/user",
   "UpstreamHttpMethod": [ "GET" ],
   "DownstreamPathTemplate": "/api/user",
   "DownstreamScheme": "http",
   "DownstreamHostAndPorts": [
    {
```

```
    "Host": "localhost",
    "Port": 7001
   }
  ],
  "FileCacheOptions": {
   "TtlSeconds": 15,
   "Region": "default",
   "Header": "OC-Caching-Control",
   "EnableContentHashing": false
  }
},

{
  "Key": "userServicewithid",
  "UpstreamPathTemplate": "/api/user/{id}",
  "UpstreamHttpMethod": [ "GET" ],
  "DownstreamPathTemplate": "/api/user/{id}",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [
   {
    "Host": "localhost",
    "Port": 7001
   }
  ],
  "RateLimitOptions": {
   "ClientWhitelist": [],
   "EnableRateLimiting": true,
   "Period": "60s",
```

```
    "PeriodTimespan": 60,
    "Limit": 10
   }
  },

  {
   "Key": "weatherforecastService",
   "UpstreamPathTemplate": "/api/weatherforecast",
   "UpstreamHttpMethod": [ "GET" ],
   "DownstreamPathTemplate": "/api/weatherforecast",
   "DownstreamScheme": "http",
   "DownstreamHostAndPorts": [
    {
     "Host": "localhost",
     "Port": 7002
    }
   ]
  }
],

"Aggregates": [
 {
  "RouteKeys": [
   "userServicewithid",
   "weatherforecastService"
  ],
  "UpstreamPathTemplate": "/api/dashboard/{id}"
 }
```

```
 ],

 "GlobalConfiguration": {
  "BaseUrl": "https://localhost:7000"
 }
}
```

Routes Section

The Routes section contains the individual route definitions. Each route maps an incoming client request to a specific downstream microservice endpoint.

User Service Route

Purpose

The userService route is used to retrieve all users from the User Service.

Configuration

```
{
 "Key": "userService",
 "UpstreamPathTemplate": "/api/user",
 "UpstreamHttpMethod": [ "GET" ],
 "DownstreamPathTemplate": "/api/user",
 "DownstreamScheme": "http",
 "DownstreamHostAndPorts": [
  {
   "Host": "localhost",
   "Port": 7001
  }
 ]
}
```

Key

- The Key property provides a unique name for the route. This key can later be referenced in aggregate routes. In this configuration, the route key is userService.

UpstreamPathTemplate

- The UpstreamPathTemplate represents the URL exposed to the client through the API Gateway. When a client sends a request to GET /api/user, Ocelot matches this route and forwards the request to the downstream service.

UpstreamHttpMethod

- The UpstreamHttpMethod property specifies which HTTP methods are allowed for this route. Since only the GET method is configured, requests such as POST, PUT, or DELETE will not be routed through this endpoint.

DownstreamPathTemplate

- The DownstreamPathTemplate defines the endpoint exposed by the downstream service. In this case, the request is forwarded to /api/user on the User Service.

DownstreamScheme

- The DownstreamScheme specifies the communication protocol used to reach the downstream service. Here, the protocol is HTTP.

DownstreamHostAndPorts

- The DownstreamHostAndPorts section specifies the location of the downstream service. Ocelot forwards the request to http://localhost:7001.

Example

When the client sends the following request:

GET /api/user

Ocelot forwards the request to:

GET http://localhost:7001/api/user

The response from the User Service is returned directly to the client.

User Service Route with ID

Purpose

The userServicewithid route is used to retrieve a specific user by providing an ID.

Configuration

```
{
  "Key": "userServicewithid",
  "UpstreamPathTemplate": "/api/user/{id}",
  "UpstreamHttpMethod": [ "GET" ],
  "DownstreamPathTemplate": "/api/user/{id}"
}
```

Route Parameter

- The {id} placeholder represents a route parameter. Ocelot extracts the value from the incoming request and automatically substitutes it into the downstream path.

Example

When the client sends:

GET /api/user/10

Ocelot extracts the value 10 and forwards the request as:

GET http://localhost:7001/api/user/10

The User Service processes the request and returns the details of the user whose ID is 10.

FileCacheOptions

Cache Manager Package

If you want to use Ocelot response caching, install:

Install-Package Ocelot.Cache.CacheManager

or

dotnet add package Ocelot.Cache.CacheManager

Purpose

Provides support for:

- **Response caching**
- **Cache expiration**
- **Cache regions**

Registration

```
builder.Services
    .AddOcelot()
    .AddCacheManager(settings =>
    {
       settings.WithDictionaryHandle();
    });
```

Purpose

- The FileCacheOptions section enables response caching. Caching improves performance by storing responses temporarily and serving repeated requests from the cache instead of calling the downstream service every time.

TtlSeconds

- The TtlSeconds property specifies how long a response remains in the cache. In this configuration, the cached response remains valid for 15 seconds.

Region

- The Region property groups cached responses under a logical category.

Header

- The Header property specifies the header used by Ocelot to manage cache behavior.

EnableContentHashing

- When set to false, Ocelot does not generate content hashes for cached responses.

Example

Suppose a client sends:

GET /api/user

Ocelot forwards the request to the User Service and caches the response for 15 seconds.

If another client sends:

GET /api/user

within the next 15 seconds, Ocelot serves the response directly from the cache without calling the User Service again.

RateLimitOptions

Purpose

- The RateLimitOptions section protects services from excessive requests by limiting how many requests a client can make within a specified period.

EnableRateLimiting

- This property enables the rate-limiting feature.

Period

- The Period property defines the rate-limiting window. A value of 60s means the request count is tracked over sixty seconds.

PeriodTimespan

- The PeriodTimespan property specifies the same duration in seconds.

Limit

- The Limit property defines the maximum number of requests allowed within the specified period.

Example

If the limit is set to **10 requests per minute and a client sends**:

GET /api/user/10

ten times within sixty seconds, all requests are processed successfully.

When the client sends the **eleventh request** within the **same sixty-second window**:

GET /api/user/10

Ocelot blocks the request and returns:

HTTP 429 Too Many Requests

Weather Forecast Service Route

Purpose

- The weatherforecastService route is responsible for retrieving weather information from the Weather Service.

Configuration

```
{
  "Key": "weatherforecastService",
  "UpstreamPathTemplate": "/api/weatherforecast",
```

```
  "DownstreamPathTemplate": "/api/weatherforecast"
}
```

Example

When the client sends:

GET /api/weatherforecast

Ocelot forwards the request to:

GET http://localhost:7002/api/weatherforecast

The response from the Weather Service is returned to the client.

Aggregates Section

Purpose

- The Aggregates section allows Ocelot to combine responses from multiple downstream services and return a single consolidated response to the client.

Configuration

```
{
  "RouteKeys": [
    "userServicewithid",
    "weatherforecastService"
  ],
  "UpstreamPathTemplate": "/api/dashboard/{id}"
}
```

RouteKeys

- The RouteKeys property specifies which routes participate in the aggregation process. In this configuration, Ocelot calls both the User Service and Weather Service.

UpstreamPathTemplate

- The aggregate endpoint exposed to clients is /api/dashboard/{id}.

Example Request Flow

When a client sends:

GET /api/dashboard/10

Ocelot extracts the value 10 from the route parameter.

It then calls the User Service:

GET http://localhost:7001/api/user/10

and calls the Weather Service:

GET http://localhost:7002/api/weatherforecast

After receiving both responses, Ocelot combines them into a single response and returns it to the client.

Example Response

```
{
  "userServicewithid": {
    "id": 10,
    "name": "John"
  },
  "weatherforecastService": {
    "temperature": 32,
    "summary": "Sunny"
  }
}
```

This allows the client to retrieve information from multiple services using a single API call.

GlobalConfiguration

Purpose

- The GlobalConfiguration section contains settings that apply to the entire API Gateway.

BaseUrl

- The BaseUrl property defines the public URL through which clients access the API Gateway.

Configuration

```
{
  "BaseUrl": "https://localhost:7000"
}
```

Example

Instead of directly calling the User Service or Weather Service, clients communicate with the API Gateway using URLs such as:

GET https://localhost:7000/api/user

GET https://localhost:7000/api/user/10

GET https://localhost:7000/api/weatherforecast

GET https://localhost:7000/api/dashboard/10

The API Gateway receives the requests, routes them to the appropriate downstream services, and returns the responses to the client.

Summary

This Ocelot configuration exposes three individual routes and one aggregate route. The User Service route retrieves all users, while the User Service with ID route retrieves a specific user based on the provided identifier. The Weather Service route retrieves weather information from the Weather microservice.

Response caching is implemented on the User Service route to improve performance by reducing repeated calls to the downstream service and serving frequently requested data from the cache.

Rate limiting is configured on the User Service with ID route to control the number of requests a client can make within a specified time period. This helps protect the service from excessive traffic, abuse, and potential denial-of-service scenarios.

The aggregate route combines responses from the User Service and Weather Service into a single response. This reduces the number of API calls that clients need to make and simplifies client-side development.

The Global Configuration defines the base URL of the API Gateway, which acts as the single entry point for all client requests. Ocelot receives incoming requests, applies configured policies such as routing, caching, and rate limiting, and then forwards the requests to the appropriate downstream services.